

Distribution Management System (DMS) for Laher Bakery

ABSTRACT

This documents encompasses the details of the entire software program including the different phases of the Software Development Life Cycle that were encountered, the system architectures, methodologies, test scripts for the functionality of the system, an appendix and all the required details needed to provide much need information for the operation of the software system.

OVERVIEW

Laher Bakery is a Bakery located at 52VH+2QP, Kenyatta Drive, Blantyre, MW. The business is listed under bakery category. It has received 2 reviews with an average rating of 2.5 stars. Their services include In-store shopping and takeout (Anon., 2023).

The Bakery bakes all its food products overnight and begins distributing them to customers and reseller posts in the early morning hours. The Bakery currently offers customers the opportunity to buy any of its food items displayed to them. The bakery operates on a large scale to meet the demands of the available location markets. The staff, including bakers and packaging personnel, work around the clock to ensure that there is fresh bread and other bakery products available for distribution. There is a bakery manager who supervises the day-to-day production before staff package it and begin allocating to specific trucks. This process involves planning routes, managing inventory and validating vehicle licences, insurance and fuel. Laher Bakery also hires a select group of drivers and driver trucks to deliver its products to its suppliers around the country. The amount of products delivered depend on the market and its customer preferences with Phalombe for example, preferring white bread over brown bread.

However, Laher Bakery have expressed concern with the current system. The manager uses a diary or a piece of paper to keep the schedule of the product orders and their information. The amount of time taken to verify each order manually has accumulated at times and has led to delays in fulfilling customer requests and has led to human errors such as incorrect data entry, lost or misplaced documents, duplicated data and misinterpretation of customer requirements. These errors have led to incorrect deliveries to customers, overproduction due to not knowing the quantity of the products as well as missed order details causing customer complaints as well as the need for rework. Laher Bakery have also expressed that they have found Difficulty in accurately calculating the expected cost of products ordered by the large number of customers they tend to on a daily basis. This has made it increasingly difficult for Laher Bakery to write receipts as they lack the knowledge of the total cost of each individual order which has led to some customers being charged more or less than their actual order. Laher Bakery also does not keep track of delivery locations for each customer. This has led to some orders being delivered to the wrong Location. It has been noted that there is difficulty when it comes to tracking the status of orders particularly with the lack of communication on the state of the customer's ordered products. This has made it very difficult for Laher Bakery to respond to any rapid ordering changes such as order cancellation. It has also made it harder to provide customers with accurate updates on their orders. Customers would also like to first hand information on when they can expect their products to be delivered to them. Laher Bakery use a paper based system for reports on any daily transaction processes. This has made it challenging for the bakery staff to handle due to the large volumes of data needed to be analyzed and increased the expense needed to maintain stationery. This has also made it difficult for the organization to monitor their distribution processes and note areas of improvement.

The Desktop application will enable the Distributors to schedule each individual orders. This will help save time as well as prevent errors and duplications in data. This will also help prevent overproduction or shortages, optimizing the use of resources allowing the Bakery. The system will also allow Laher Bakery to automatically generate quotations of the products being distributed to each consumer. This will help Distributors know the cost of the products

they are distributing. The distribution system will also have a geolocation feature that will allow users to provide a designated delivery Location so that Laher Bakery is aware of the specific consumer address the products are to be distributed. This will prevent Laher Bakery from distributing the products to the wrong location. The application will also allow for real-time tracking of the orders placed by consumers. This will help Laher Bakery in to inform the customer of the real time status of their orders by not only showing the state of their orders but also by sending a message to their user account. The system will generate a report of all the delivery processed occurred. This will reduce the workload on the employees, reduce the cost incurred to provide paper as well as help Laher Bakery monitor and analyze their distribution processes and identify areas of improvement.

2. AIMS, OBJECTIVES, SCOPE AND SYSTEM ARCHITECHITURE

2.1. AIM

The aim of this desktop application is to help Laher Bakery in scheduling their orders, generate quotations of their products in each delivery, allow Geolocation of addresses, allow customers to track the status of their orders and send notifications to customers on the real time status of their expected deliveries and generate a report on each distribution.

2.2. OBJECTIVES

2.2.1. Investigating and Analyzing the Current System

- 2.2.1.1. Conducting Interviews.
- 2.2.1.2. Analyzing collected information
- 2.2.1.3. Investigating existing systems with similar functional requirements.
- 2.2.1.4. Create rich picture of current system
- 2.2.1.5. Conducting Technological review
- 2.2.1.6. Prioritizing work by organizing and arranging content and activities in a hierarchy based on importance for a more efficient approach using the MOSCOW prioritization.

2.2.2. Designing the System

- 2.2.2.1. Designing User Interface with visual designing tools to meet the requirements of a good user interface. This will reference the Design principles presented in the Information System Analysis module.
- 2.2.2.2. Designing a Class Diagram using Unified Modelling Language (UML). The class diagram will provide a visual model of the static view showing different parts of the system and how they interact with each other.
- 2.2.2.3. Designing Use case. This will show how users interact with the system.
- 2.2.2.4. Database Design. This will involve using proper database management techniques to allow the system software to function properly.

2.2.3. Developing the System

- 2.2.3.1. Developing an actual interface based off of the visual designs created in the design phase. . This will reference the Design principles presented in the Information System Analysis module.
- 2.2.3.2. Developing Digital Prototyping. Creating the first versions of the system.
- 2.2.3.3. Converting the class diagram from the design phase into an actual working code. The code will follow the visualized architecture presented in the class diagram.
- 2.2.3.4. Developing a database that will handle the storage and management of Laher Bakery's data.

2.2.4. Testing and Implementing the System

- 2.2.4.1. Black Box Testing. This will be done by inserting data and checking to see if the system provides the necessary input and output. Test cases, test scenarios and test scripts will be outlined to conduct proper testing.
- 2.2.4.2. White Box Testing. This will test the inner working code of the system.
- 2.2.4.3. User Acceptance Testing. This will be done by inserting data and checking to see if the system does what it is required to do. This will involve end users conducting processes on the system and giving feedback on its strengths and available weaknesses.
- 2.2.4.4. Boundary Testing. This test will be conducted to test the system to check areas where errors may creep into the system. This will be done to see the systems error handling capabilities of the system. This type of test will query the system to see its limitations in the conducting of its processes.

2.2.5. Documentation

- 2.2.5.1. Creating a user manual for the proposed system that details the specifics and directions of how to effectively use the developed system.
- 2.2.5.2. Creating a Technical Manual. This will involve creating a well detailed explanation on the technical aspects of the newly developed system.

3. ANALYSIS

Analysis is a phase in the Systems Development Life Cycle (SDLC) that involves collecting and interpreting information facts, identifying problems and decomposition of a system into its components to define the detailed functional user requirements of the system software so that they can be addressed (www.tutorialspoint.com, 2023). The analysis phase is important because it allows system analysts to receive feedback and support from relevant internal and external stakeholders. This feedback allows system analysts to understand the problems that need to be addressed within a system (Team, 2021).

However, gathering of Information is required to analyze and Information system and its resources. This involves employing information gathering methods which can be defined as repeated processes that are used to create and organize data across different kind of sources (projectvictor.com, 2023). There are several information gathering methods such as surveys, interviews, questionnaires and observations.

Interviews were the Information gathering method of choice used. This helped save the cost needed to gather information from each individual. Interviews also helps build trust. Face-to-face communication between the interviewer and the interviewee helps put the correspondents at ease and build mutual relations and understanding which in turn will make the interviewees trust more. (sg.bestessaywriting.com, 2023).

3.1. Requirements

3.1.1. Functional Requirements

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks (www.altexsoft.com, 2021). These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product (chitrasingla2001, 2022).

i) Order Placement

The system must allow the customer to automatically place an order by providing the necessary details such as selecting the products and quantity.

ii) Quotation generation

The system must automatically generate quotation of those products to allow users to be able to know the cost of the products.

iii) Geolocation

When customers are ordering, the system must allow them to provide a user address at which they would like the products to be delivered.

iv) Order Scheduling

The system must allow the distributor to schedule the customer orders to be fulfilled allowing the company to prevent overproduction or shortages.

v) Order Processing

The system must allow the distributor to sort and compile each order depending on each customer's needs and destination zones.

vi) Expected Delivery Notification

The system should allow the distributor to inform the customer when the product can be picked up by sending a message to their user account.

vii) Real time Order Tracking

When the product has been packaged, the system must allow customers to check on the real-time status of their orders to show whether their orders.

viii) Order Termination

The system should allow the customer to cancel the order of their products.

ix) Order Delivery Verification

The system must allow the customer to verify if the customer has successfully received their ordered products.

x) Invoice Generation

The system must allow the distributor to come up with a payment bill for the ordered products.

xi) Report Generation

The system must allow the distributor to come up with a logistical report of the distribution process including approved, rejected or pending processes.

3.1.2. Non-Functional Requirements

Non-functional requirements in software engineering refer to the characteristics of a software system that are not related to specific functionality or behavior (AshmeetSaggu, 2023). They describe how the system should perform, rather than what it should do. Non-Functional requirements explain the constraints and limitations of the system to be designed (visuresolutions.com, 2023).

i) Usability

The system must have a user-friendly interface that is easy to navigate and requires minimal technical training for both customers and distributors

ii) Availability

The system must be readily available for use, allowing customers to place orders and track their status.

iii) Scalability

The system should be able to accommodate growth in the number of customers and orders.

iv) Security

The system must not grant users access to other accounts other than their own.

v) Compatibility

The system must be capable of running on various device platforms to ensure a seamless experience for the users.

vi) Maintainability

The system must be designed and developed using modular and maintainable code, allowing for easy updates, error handling and system enhancements.

3.2. Use case

A use case is a concept used in software development, product design, and other fields to describe how a system can be used to achieve specific goals or tasks. It outlines the interactions between users or actors and the system to achieve a specific outcome (Daly, 2022).

Note: Use case description is available in the appendix.

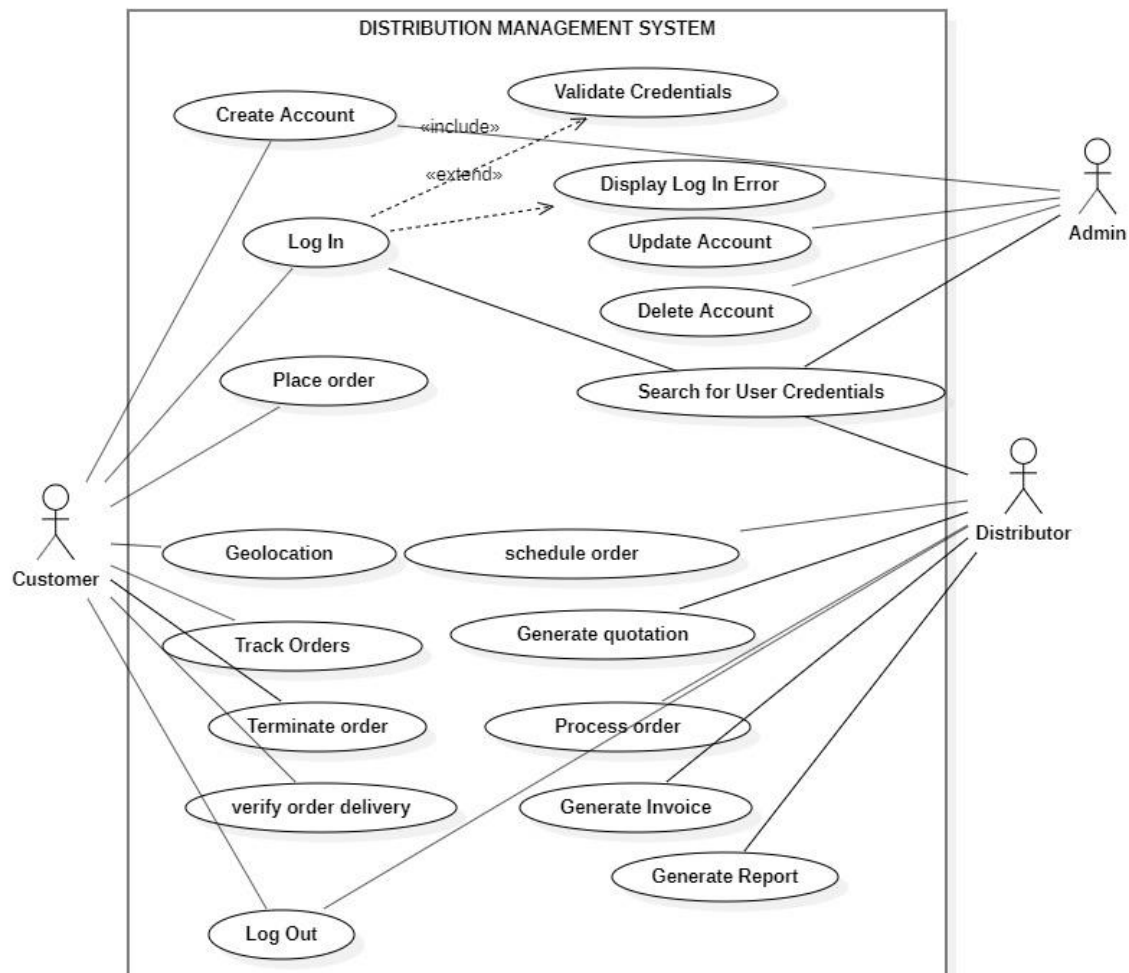


Figure 1. shows a use case diagram that models the requirements of the system. Diagram generated by Star UML

3.3. SYSTEM ARCHITECTURE

Software architecture is the high-level design and organization of software in which we make important decisions regarding its overall structure, such as the relationships between components, data flow patterns, and the mechanism for communication between different parts of the system (MANCHANDA, 2023).

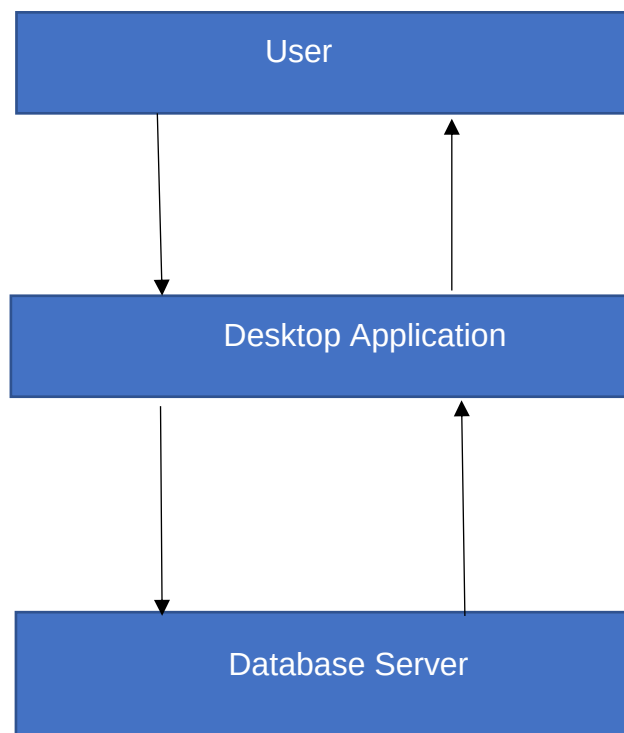


Figure 2. shows a architecture diagram that models the requirements of the system.

Desktop Platform

This is the physical device where the system platform will run. It is responsible for windowing, operating systems, input and output handling, file access and device management which are all used by the system software during execution.

Desktop Application

When the user first accesses the system, they will view the windows forms, buttons and other necessary widgets of the user interface. The application translates and formats data into a standardized format that can be understood by different systems.

Database Server

Data is sent into a database which handles the management of different forms of data that are transmitted between adjacent devices within the system.

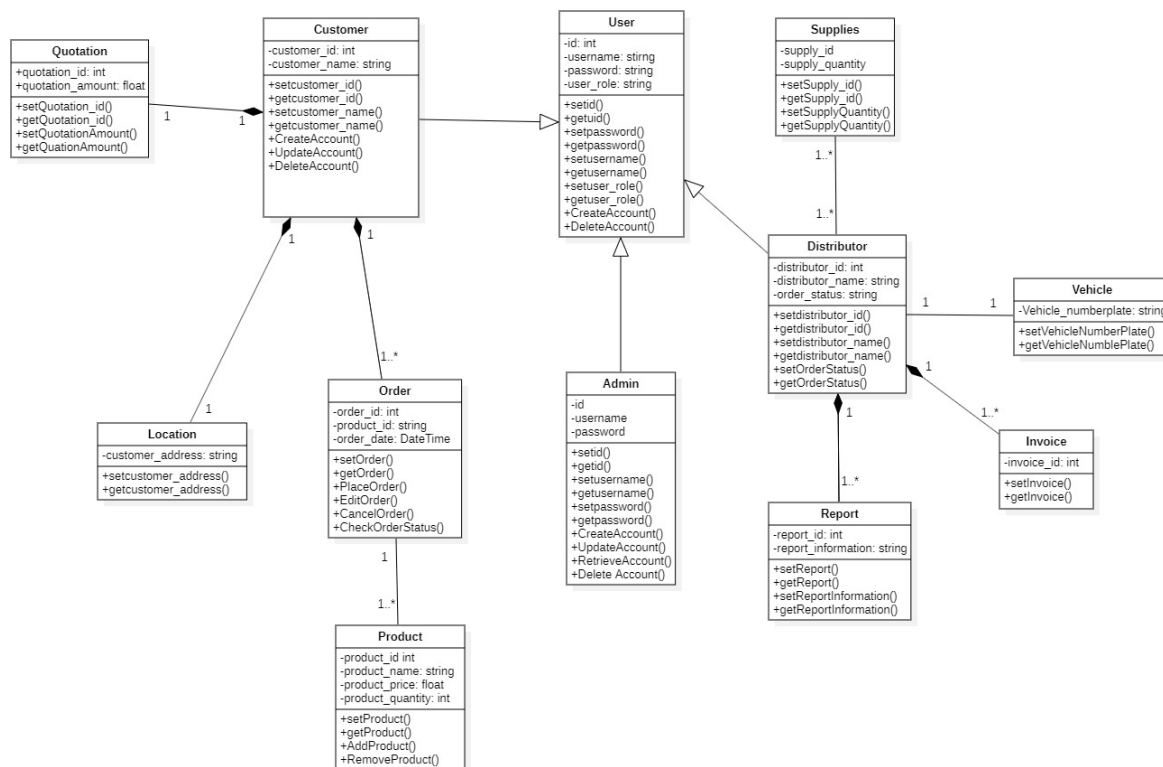
4. DESIGN

4.1. STRUCTURAL DIAGRAM

A structure diagram is a conceptual modeling tool used to document the different structures that make up a system such as a database or an application. It shows the hierarchy or structure of the different components or modules of the system and shows how they connect and interact with each other. It is a tool used to guide developers to ensure that all parts of the system work as intended in relation to all the other parts (Rouse, 2014).

CLASS DIAGRAM

Class diagrams are the blueprints of your system or subsystem. You can use class diagrams to model the objects that make up the system, to display the relationships between the objects, and to describe what those objects do and the services that they provide (www.ibm.com, 2021).



Note: Description of class diagram elements is available in the document appendix

The class above provides two different kinds of users within the system. The Customer will provide their username and password and will be automatically assigned an id at the point of registration. They will be able to order a particular number of products of their choice and provide their address through the desktop software's geolocation feature. The software will allow customers to edit contents of their orders, cancel their orders and Check the status of their orders.

Once, the distributors receive an order, the system will check to see if the supplies needed to fulfill the order are available. The supplies will determine whether the distributor can accept or reject a particular customer order. If the order is accepted, the software will automatically generate a quotation of the order. As the bakery produces the items, the distributor will be able to notify the customers of the status of their orders. Once the items are produced, the distributor will assign a particular vehicle in which the orders will be distributed. The software will then generate an invoice and send it to the customer. Finally, the software will generate a report of the transactions.

4.2. BEHAVIORAL DIAGRAM

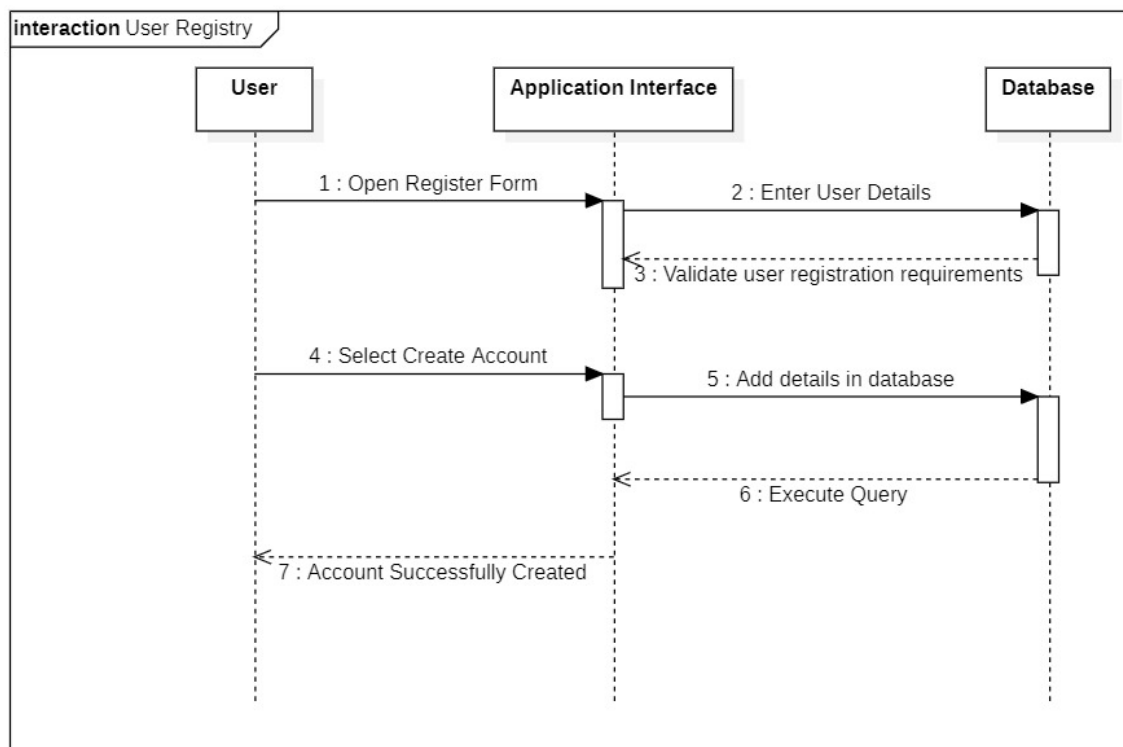
Behavioral diagrams are used to visualize, specify, construct, and document the dynamic aspects of a system. It shows how the system behaves and interacts with itself and other entities (users, other systems). They show how data moves through the system, how objects communicate with each other, how the passage of time affects the system, or what events cause the system to change internal states (www.visual-paradigm.com, 2023).

SEQUENCE DIAGRAMS

UML Sequence diagrams are interaction diagrams that detail how operations are carried out. They capture the interaction between objects in the context of a collaboration. Sequence diagrams are time focus and they show the order of the interaction visually by using the vertical axis of the diagram to represent the time what messages are sent and when (www.visual-paradigm.com, 2023).

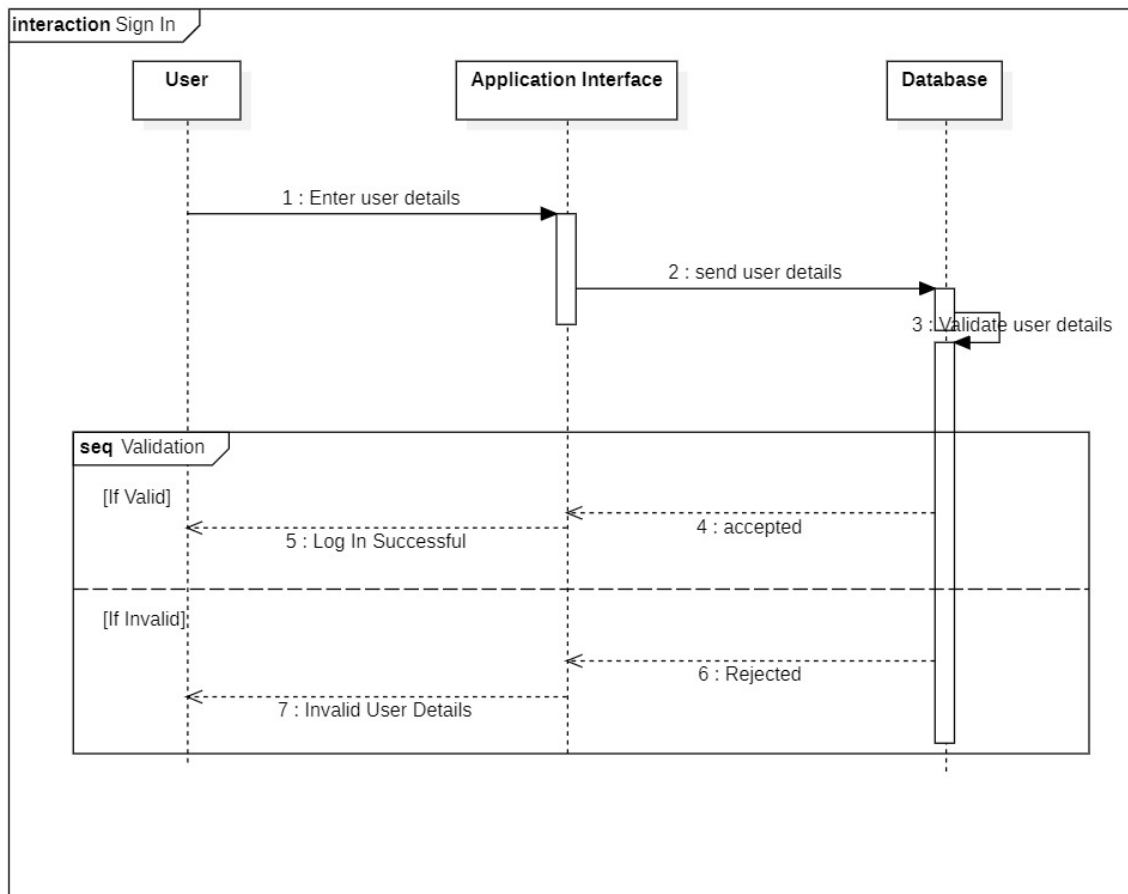
Note: Description of sequence diagram elements is available in the document appendix

User Registry



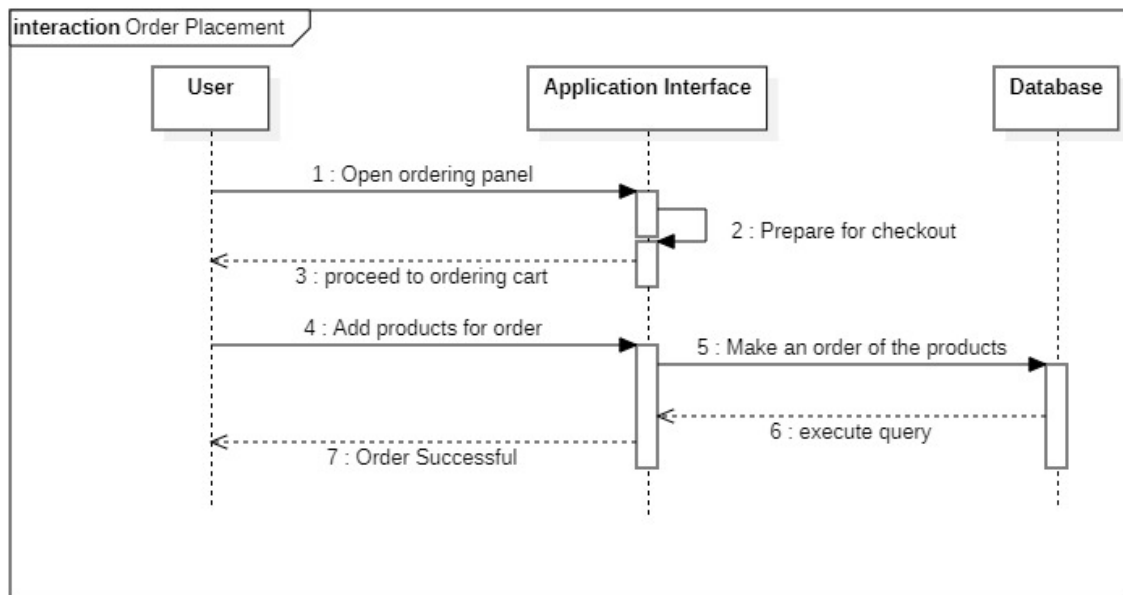
The user will interact with the application user interface to enter user details (username, password, user roles) and the database will validate to see if the user registration requirements. The user will click on the select account button which will prompt to add details into the database and once the query is executed, the application will notify the user that their account has been successfully created.

Sign Up



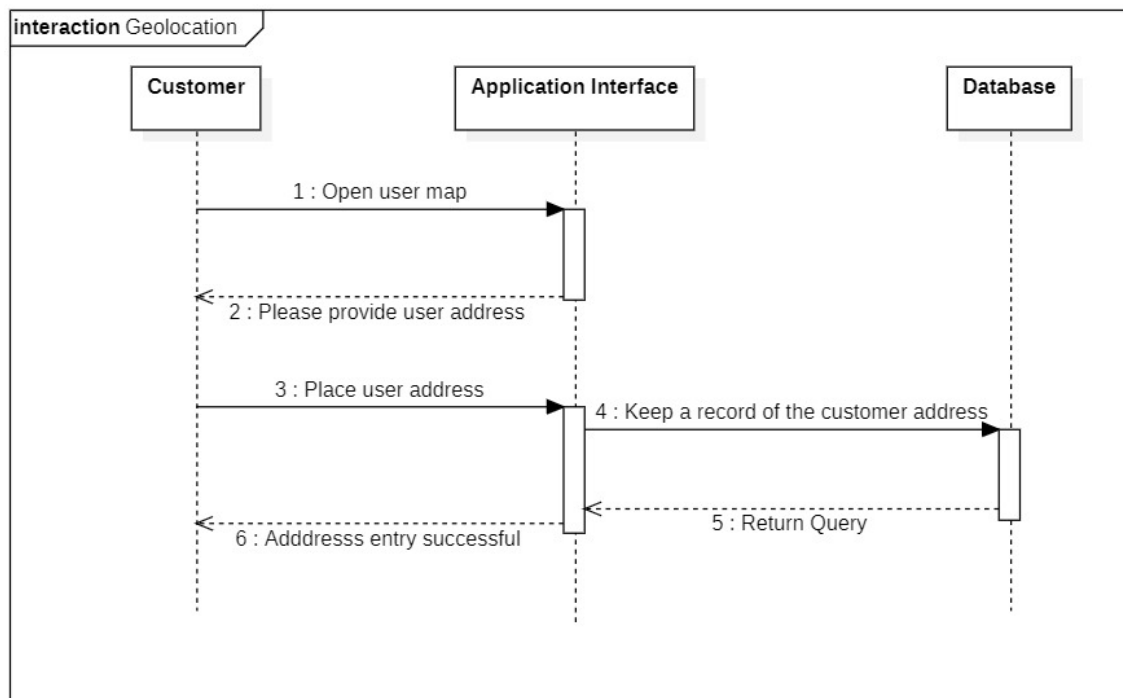
When the user enters their details in the entry fields of the application, the applications sends their details to the software's database. The database runs a query that validates user details. If the details are valid, the database will accept them and display a message that tells the user their log in was successful. If the credentials are invalid, the application.

Order Placement



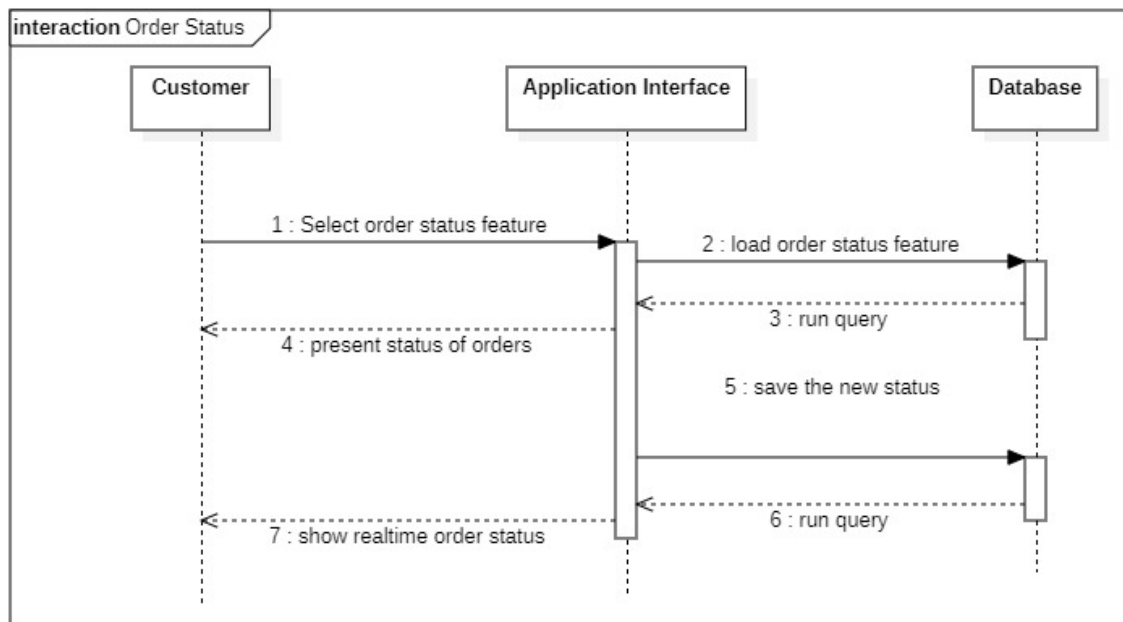
The customers will open an ordering user interface panel that will automatically prepare for checking out by allowing them to place the orders through an ordering card. The software will then proceed to allow them to add products for an order which will prompt the database to create a record of an order occurring within the system. The database will finally execute the query to say the order was successful.

Geolocation



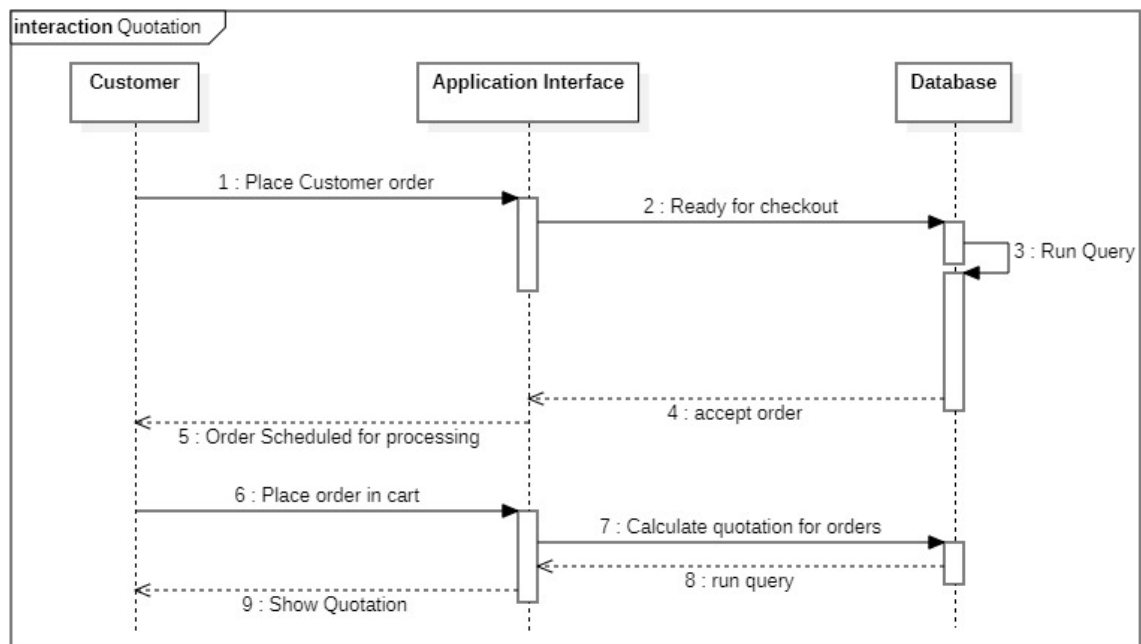
When the user, customer taps the location feature, they will be quickly redirected to a map page that will ask them to please provide their user address. Once the user adheres to the instruction, the software will run a query that will keep a record of the customer address before it finally returns a message that says the address was entered successfully.

Real Time Order Status Tracking



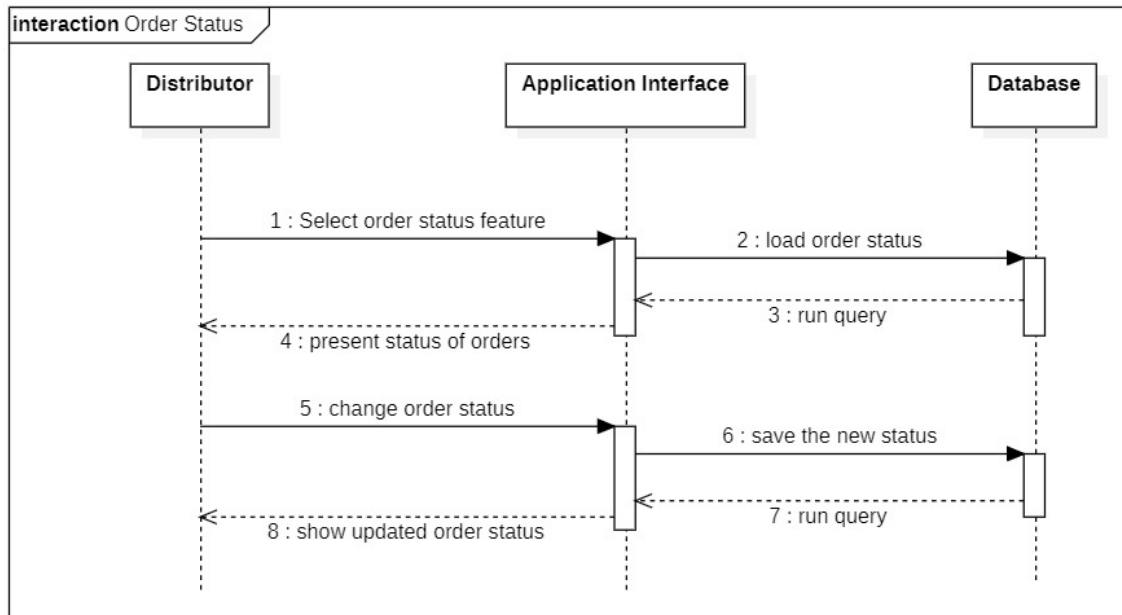
Once the customer selects the order status, the application loads and runs a query that retrieves the real time status transmitted by the distributor, the new status is saved in a database before being finally presented to the customer.

Generate Quotation



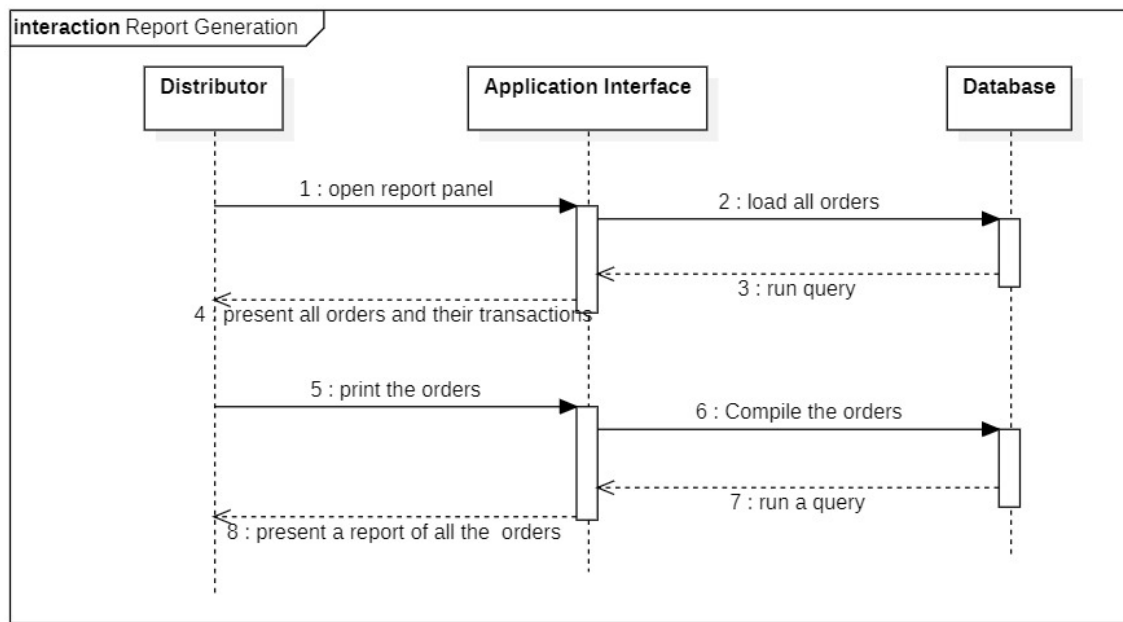
When the customer places an order, the application will ask the database to prepare for checkout. This will cause the database to run and execute a query that will accept the order and store its details in the database before scheduling the order for processing. The order will automatically be placed in the customer cart and the application will make the necessary calculations needed to get the quotation. The database will then finally run a query that retrieves the calculated quotation before the application displays to the user through the application interface

Assign Order Status



The Distributor will select the order status feature to load the status of orders present in the database. The database will then run a query that retrieves data from the database and presents the real time status of orders to the distributor. The Distributor then will change the order status. The database will once again run a query that saves the new order status before present the distributor with an updated order status.

Generating Report



The Distributor will select the option to automatically generate the report and the database will run a query that retrieves all the orders and present them to the distributor. The Distributor will select the option to print order which will make the software compile the information regarding the orders. Finally, it will run a query that will generate the report and present it to the distributor. The report will contain all the information collected during transactions including pending, approved, canceled and rejected orders.

4.3. Justification of Design Approach

Class diagram: To be more specific, a class diagram in the Unified Modeling Language (UML) is a static structure diagram that describes the structure of a system by representing the system's classes, their attributes, operations, and the relationships between objects (Kondratowicz, 2023).

Sequence diagram: Sequence diagrams are great tools at the start of a project as it shows the user what has to happen in a step-by-step fashion (Ghoshal, 2004).

5. CONCLUSION

This document elaborated the details that were followed in the development of the software program. The conceptual stage of software development required assessing a target organization and gathering information on the organizations in terms of the organizations daily operations, company stakeholders, the problems the organizations faced and possible solutions to the problems faced by the organization. The proposal phase then involved providing a through breakdown of how the software being advertised would be developed. The analysis phase involved evaluating the various user requirements and how those would fit into the software through architectural and class diagrams. Finally, the design phase provided a simple design flow of how different components of the software would interact through the structural diagram and how they would behave through the behavioral diagram.

EVALUTION OF WORK

This documents aims to describe how a distribution management system may help in managing the proper transportation of goods from the company to the customers who live in various places.

The requirements that were put in place when developing the software in question have successfully been met. Customers are able to place their orders in the cart, they are able to generate a quotation of their ordered products. Once they prepare the products for checkout, the customers are also expected to provide their user address through the geolocation feature in the system. Distributors then can schedule and process the various orders and when they are doing so, the customers get to track their orders in real time. Distributors can send messages to notify the customers of their orders. When customers verify the arrival of their products, the orders are deemed to be completed and the Distributor can finally generate a report of the transactions.

However, despite its completion the system may still undergo future developments. The system may allow customers to pay for their orders through the application in the future which would allow faster transactions to occur.

6. REFERENCES

Anon., 2023. [Online]

Available at: <https://zaubee.com/biz/bake-land-bakery-gpsmlot0>

AshmeetSaggu, 2023. www.geeksforgeeks.org. [Online]

Available at: <https://www.geeksforgeeks.org/non-functional-requirements-in-software-engineering/>

chitrasingla2001, 2022. www.geeksforgeeks.org. [Online]

Available at: <https://www.geeksforgeeks.org/functional-vs-non-functional-requirements/>

Daly, N., 2022. www.wrike.com. [Online]

Available at: <https://www.wrike.com/blog/what-is-a-use-case/#What-is-a-use-case-Use-cases-explained>

MANCHANDA, A., 2023. www.netsolutions.com. [Online]

Available at: <https://www.netsolutions.com/insights/why-software-architecture-matters-to-build-scalable-solutions/>

projectvictor.com, 2023. projectvictor.com. [Online]

Available at: <https://projectvictor.com/knowledge-base/information-gathering-techniques/#:~:text=Managing%20information%20involves%20gathering%20and,across%20different%20kinds%20of%20sources.>

Rouse, M., 2014. www.techopedia.com. [Online]

Available at: <https://www.techopedia.com/definition/22358/structure-diagram>

sg.bestessaywriting.com, 2023. bestessaywriting.com. [Online]

Available at: <https://sg.bestessaywriting.com/blog/advantages-disadvantages-interviews-research/>

Team, S., 2021. svitla.com. [Online]

Available at: <https://svitla.com/blog/system-development-life-cycle#:~:text=The%20analysis%20phase%20in%20SDLC,your%20potential%20users%20will%20be.>

visuresolutions.com, 2023. visuresolutions.com. [Online]

Available at: <https://visuresolutions.com/blog/functional-requirements/>

www.altexsoft.com, 2021. www.altexsoft.com. [Online]

Available at: <https://www.altexsoft.com/blog/business/functional-and-non-functional-requirements-specification-and-types/#:~:text=Functional%20requirements%20are%20product%20features,system%20behavior%20under%20specific%20conditions.>

www.ibm.com, 2021. www.ibm.com. [Online]

Available at: <https://www.ibm.com/docs/en/rsm/7.5.0?topic=structure-class-diagrams>

www.tutorialspoint.com, 2023. www.tutorialspoint.com. [Online]

Available at:

https://www.tutorialspoint.com/system_analysis_and_design/system_analysis_and_design_o_verview.htm#:~:text=System%20analysis%20is%20conducted%20for,what%20the%20syste

m%20should%20do.

[Accessed 26 June 2023].

www.visual-paradigm.com, 2023. *www.visual-paradigm.com*. [Online]
Available at: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/behavior-vs-structural-diagram/>

www.visual-paradigm.com, 2023. *www.visual-paradigm.com*. [Online]
Available at: <https://www.visual-paradigm.com/guide/uml-unified-modeling-language/what-is-sequence-diagram/>

7. APPENDIX

7.1. Requirements Collection Catalog

Functional Requirements

Functional requirements are product features or functions that developers must implement to enable users to accomplish their tasks (www.altexsoft.com, 2021). These are represented or stated in the form of input to be given to the system, the operation performed and the output expected. They are basically the requirements stated by the user which one can see directly in the final product (chitrasingla2001, 2022).

xii) Order Placement

The system must allow the customer to automatically place an order by providing the necessary details such as selecting the products and quantity.

xiii) Quotation generation

The system must automatically generate quotation of those products to allow users to be able to know the cost of the products.

xiv) Geolocation

When customers are ordering, the system must allow them to provide a user address at which they would like the products to be delivered.

xv) Order Scheduling

The system must allow the distributor to schedule the customer orders to be fulfilled allowing the company to prevent overproduction or shortages.

xvi) Order Processing

The system must allow the distributor to sort and compile each order depending on each customer's needs and destination zones.

i) Order Placement

The system must allow the customer to automatically place an order by providing the necessary details such as selecting the products and quantity.

ii) Quotation generation

The system must automatically generate quotation of those products to allow users to be able to know the cost of the products.

iii) Geolocation

When customers are ordering, the system must allow them to provide a user address at which they would like the products to be delivered.

iv) Order Scheduling

The system must allow the distributor to schedule the customer orders to be fulfilled allowing the company to prevent overproduction or shortages.

v) Order Processing

The system must allow the distributor to sort and compile each order depending on each customer's needs and destination zones.

vi) Expected Delivery Notification

The system should allow the distributor to inform the customer when the product can be picked up by sending a message to their user account.

vii) Real time Order Tracking

When the product has been packaged, the system must allow customers to check on the real-time status of their orders to show whether their orders.

viii) Order Termination

The system should allow the customer to cancel the order of their products.

ix) Order Delivery Verification

The system must allow the customer to verify if the customer has successfully received their ordered products.

x) Invoice Generation

The system must allow the distributor to come up with a payment bill for the ordered products.

xi) Report Generation

The system must allow the distributor to come up with a logistical report of the distribution process including approved, rejected or pending processes.

.

xii) Order Termination

The system should allow the customer to cancel the order of their products.

xiii) Invoice Generation

The system must allow the distributor to come up with a payment bill for the ordered products.

xiv) Report Generation

The system must allow the distributor to come up with a logistical report of the distribution process including approved, rejected or pending processes.

a) Non-Functional Requirements

Non-functional requirements in software engineering refer to the characteristics of a software system that are not related to specific functionality or behavior (AshmeetSaggu, 2023). They describe how the system should perform, rather than what it should do. Non-Functional requirements explain the constraints and limitations of the system to be designed (visuresolutions.com, 2023).

vii) Usability

The system must have a user-friendly interface that is easy to navigate and requires minimal technical training for both customers and distributors

viii) Availability

The system must be readily available for use, allowing customers to place orders and track their status.

ix) Scalability

The system should be able to accommodate growth in the number of customers and orders.

x) Security

The system must not grant users access to other accounts other than their own.

xi) Compatibility

The system must be capable of running on various device platforms to ensure a seamless experience for the users.

xii) Maintainability

The system must be designed and developed using modular and maintainable code, allowing for easy updates, error handling and system enhancements.

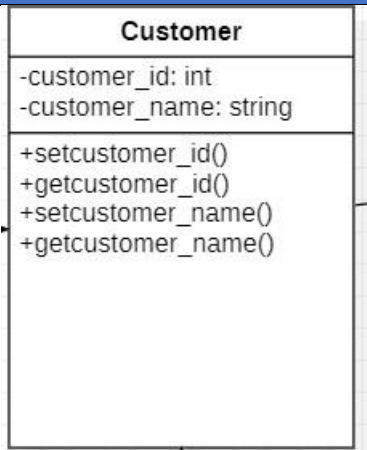
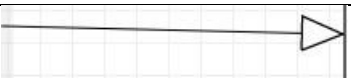
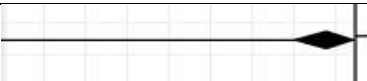
7.2. Use Case Description

Use Case	Description
Create Account	This use case describes the process of user registry into the system.
Log In	This use case describes the process of user authentication into the system. This will involve checking if the user's details are valid and displaying a log in error in invalid.
Place order	This use case describes the process of a customer placing an order for products in the system.
Geolocation	This use case describes the process of a customer providing their geographical location address so the products can be delivered.
Track order	This use case describes the process of both customers and distributors being able to track orders.
Terminate order	This use case describes the process of a customer cancelling an order for products in the system.
Verify order	This use case describes the process of a customer verifying if the order has been successfully completed in the system.
Generate Quotation	This use case describes the process of a distributor calculating the cost of the customer's ordered products.
Schedule order	This use case describes how the distributor can select the orders that can be fulfilled.
Process order	This use case describes the distributor sorting and compiling each order depending on each customer's needs and destination zones.
Generate Invoice	This use case describes the distributor generating a payment bill for the ordered products.
Generate Report	This use case describes the distributor generating a logistical report of the distribution process including approved, rejected or pending processes.
Log Out	This use case describes the process of the user leaving their account

7.3. Class diagram descriptions

Classes	Class Description
User	This class contains username and password attributes that will be used to create accounts in the system. It also contains an ID attribute that will be automatically assigned to every new user. Every user has a specific role in the system's database
Customer	This is a role for a user in the system. The customer class inherits some attributes of the user class as such they can create accounts.
Admin	This is a role for a user in the system. The customer class inherits some attributes of the user class as such they can create accounts
Order	This class allows the user to place orders of everything they want. It is entirely depended on the customer choosing they type of order they want. The order class also has methods that allow customers to edit orders as well cancel orders. It has a composition relationship meaning the orders are dependent on the customer.
Products	This class allows the Customer to decide the type of product they would like to order as well as the price of those ordered products and the quantity of those ordered products. The pro The class also has the product_id attribute which uniquely identifies each product ordered.
Location	This class allows the users to provide their address where the products might me distributed. It has a composition relationship meaning the location is depended on the customer.
Distributor	This class is a role for another type of user in the system. This class inherits some of the attributes of the user class allowing to be capable of creating accounts in the system. The distributor can also change the status of the orders during products.
Supplies	This class represents the supplies that will be used to ccreate the orderin products. This has an association with the distributor class because once the order is received, the software must automatically check if the supplies needed to fulfil the order and notify

	the distributor so that they can decide whether to approve or reject it.
Quotation	This class automatically calculate the amount of expected expenses that are generated by each customer order. This will only occur if the Distributor approves an order. It was a composition relationship meaning the quotation is dependent on the customer placing the order.
Vehicle	This class is initiated when the order has been fulfilled. This class will allow the distributor to allocate the products to a specific transportation vehicle.
Invoice	This class allows the distributor to generate invoices of all the processed orders.
Report	This class allows the distributor to generate logistics of the daily orders and present it to the distributor through the user interface.

Class Elements	UML Notation	Description
Class	 <pre> classDiagram class Customer { -customer_id: int -customer_name: string +setcustomer_id() +getcustomer_id() +setcustomer_name() +getcustomer_name() } </pre>	A class represents a static model of a software. They are used as a visual description of a particular software that describes how different elements within the software operate together. It contains attributes that are the characteristics of the class and they also contain methods which are the actions that the particular class will be performing within the system.
Relationships		This UML notation is called Inheritance. It denotes that one class inherits specific attributes and methods from another class. The class that possesses the attributes and methods is called a superclass and the one that inherits it is called a sub class
		This UML notation is called a composition. It denotes that particular class's existence is entirely dependent on another class

		This UML notation is called an association. It denotes that one particular class will directly interact with another class within the system.
--	---	--

7.4. Test Scripts

7.4.1. Black Box Testing

Unit Test 1	Objective: User Authentication into the system			
Test Case	Description	Task	Expected Result	Actual Result
1.1 Admin Login	Log in as admin into the system	Administrators can log into the system with credentials Username = ADMIN Password = ROOT	Admin can login to their dashboard	Admin can login to the dashboard
1.2 Customer Login	Log in as Customer into the system	Customers who have accounts can login	Customers who have accounts can login to the ordering page	Customers who have accounts can login to the ordering page
1.3 Distributor Login	Log in as Distributor into the system	The Distributor can login by entering credentials Username = DISTRIBUTOR Password = ROOT	The Distributor can login to their dashboard	The Distributor can login to their dashboard

Unit Test 2	Objective: Order Placement			
Test Case	Description	Task	Expected Result	Actual Result
2.1. Selecting products	Customers will select the products they would like to order.	Customers will click on the drop down combo box and the just to the left menu bar	The drop down menu should provide a catalogue of all the products available for ordering to them	The drop down menu should provide a catalogue of all the products available for ordering to them
2.2. Allocating Products to the shopping cart	Customers will be able to allocate products to a shopping cart	Customers will have to select the quantity of that particular product they would like to order	The allocation of customers to cart will provide the cost of the various products and the sum total of the transaction	The allocation of customers to cart will provide the cost of the various products and the sum total of the transaction

Unit Test 3	Objective: Quotation Generation			
Test Case	Description	Task	Expected Result	Actual Result
3.1 Generate Quotation	Users must be given the option of being able to print the bill of the orders they would like to purchase	Customer must click on the print quotation button represented by receipt cycle within the transaction section of the window	A new save window will appear to save the quotation	A new save window will appear to save the quotation
3.2 Quotation Location	Users will choose the location they would like to save the quotation	Users will do so by browsing through the computer menu	The windows will change according to the location they want to save the generated quotation	The windows will change according to the location they want to save the generated quotation
3.3 Save Quotation	Users will be able to Save the quotation	Users will press the save quotation once they have decided the document location	Generate quotation document	Generate quotation document

Unit Test 4	Objective: Geolocation			
Test Case	Description	Task	Expected Result	Actual Result
4.1 Search Location	Customers search for their real tie location	Customers search for their location on the map by typing their geographical location on the search bar provided.	Customers successfully find their geolocation	Customers successfully find their geolocation
4.2 Pin location	Customers pin their location on the map.	Customers hover the customer on the map and click on the exact point they are on to pin their location	Customer location is retrieved with latitude and longitude displaying to validate their position	Customer location is retrieved with latitude and longitude displaying to validate their position
4.3 Send Location	Send customer address to distributor	Customer's press on the send location button represented by the location icon to send their geographical location	Customer Geographical address is sent to complete the order	Customer Geographical address is sent to complete the order

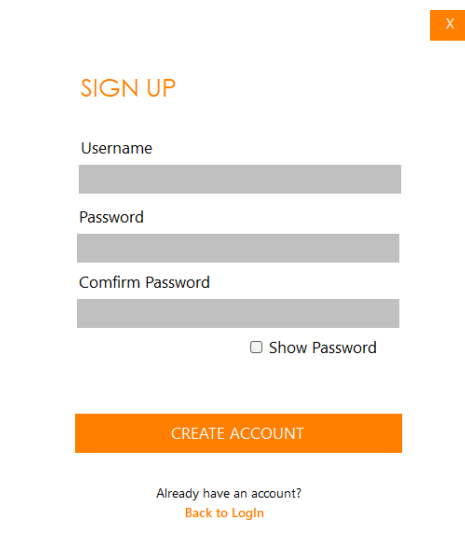
Unit Test 5	Objective: Message Notification			
Test Case	Description	Task	Expected Result	Actual Result
5.1 Send Message	Distributors send a message to confirm the transportation of the ordered products	Distributor send a notification message	Send Message	Message Sent Successfully
5.2. Receive Message	Customers received message notification	Customer Receives Notification	Receive Message	Message Received Successfully

Unit Test 6	Objective: Real time Order Status			
Test Case	Description	Task	Expected Result	Actual Result
6.1 View Orders	Distributor can view the orders depending on where the operations are at the bakery	When a distributor login, the can click the orders, tab button which will allow them to view all the orders that	View Orders	Orders Viewed Successfully
6.2 Change Order Status	Distributors can change the status of customer orders	The distributor has a access to a drop down menu which allows them to change the status of customer orders	Change Order Status	Order Changed
6.3 Track Orders	Customers can track the status of their orders in real time	When the customers goes to their orders page, they can view the status of their orders in real time	Real time order status	Real time order status
6.4 Verify Reception of products	Users can verify if they have received their ordered products	Users verify if they have received their orders by clicking the button available to them in their orders menu tab	Verified Order Reception	Order Verified and Completed

Unit Test 7	Objective: Generate Report			
Test Case	Description	Task	Expected Result	Actual Result
7.1	Distributor can generate a report of all the transactions	When a distributor clicks the generate report button the system creates a report of all the processes	Generate Report	Report Generated Successfully;

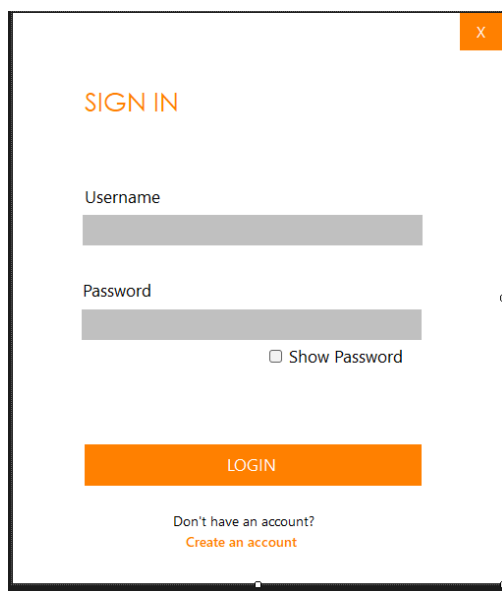
7.5. User guide

Customer



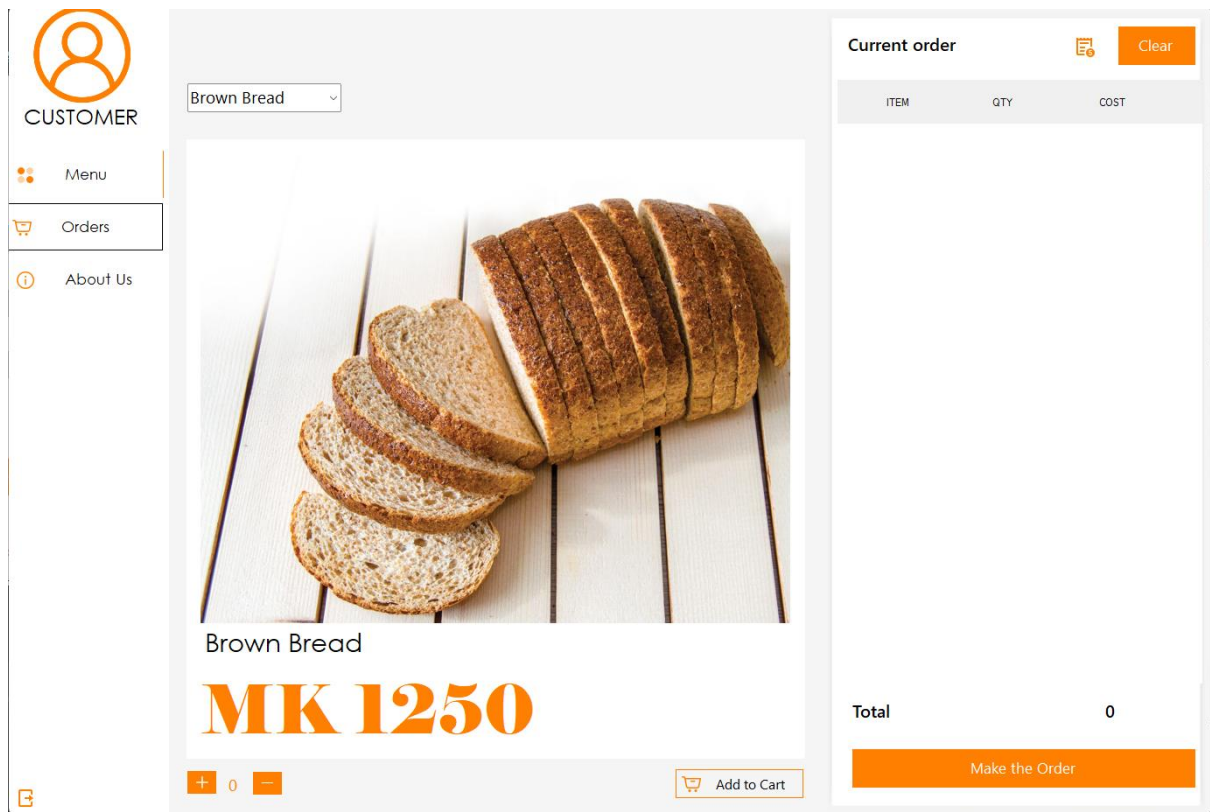
A screenshot of a 'SIGN UP' form. The form is titled 'SIGN UP' in orange. It contains three input fields for 'Username', 'Password', and 'Confirm Password'. Below the 'Password' field is a checkbox labeled 'Show Password'. At the bottom is an orange button labeled 'CREATE ACCOUNT'. Below the button is a link 'Back to Login' in orange. The form is enclosed in a white box with a thin black border and a close button (X) in the top right corner.

When the program runs, the first window that users are presented with is the login page which will allow customers to create their accounts to access the system. The system will demand that customers use strong password so as to protect their accounts.

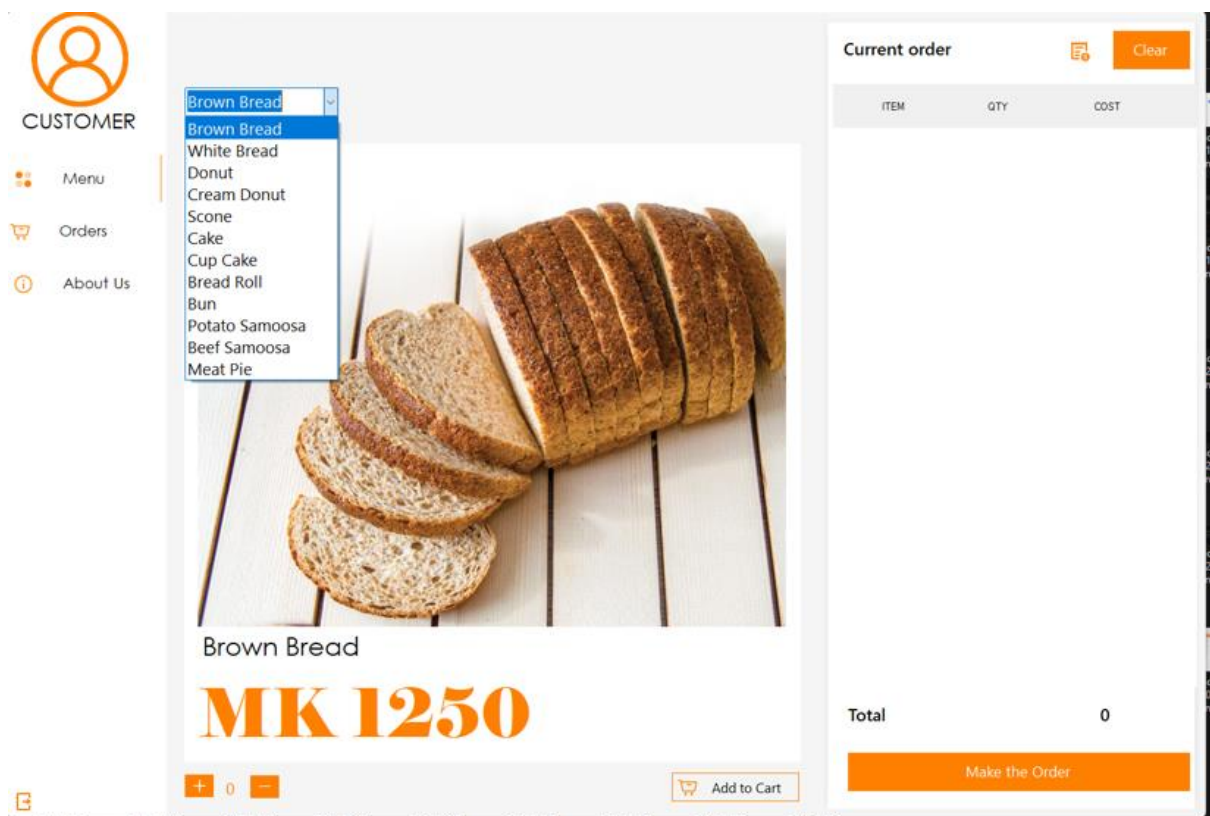


A screenshot of a 'SIGN IN' form. The form is titled 'SIGN IN' in orange. It contains two input fields for 'Username' and 'Password'. Below the 'Password' field is a checkbox labeled 'Show Password'. At the bottom is an orange button labeled 'LOGIN'. Below the button is a link 'Create an account' in orange. The form is enclosed in a white box with a thin black border and a close button (X) in the top right corner.

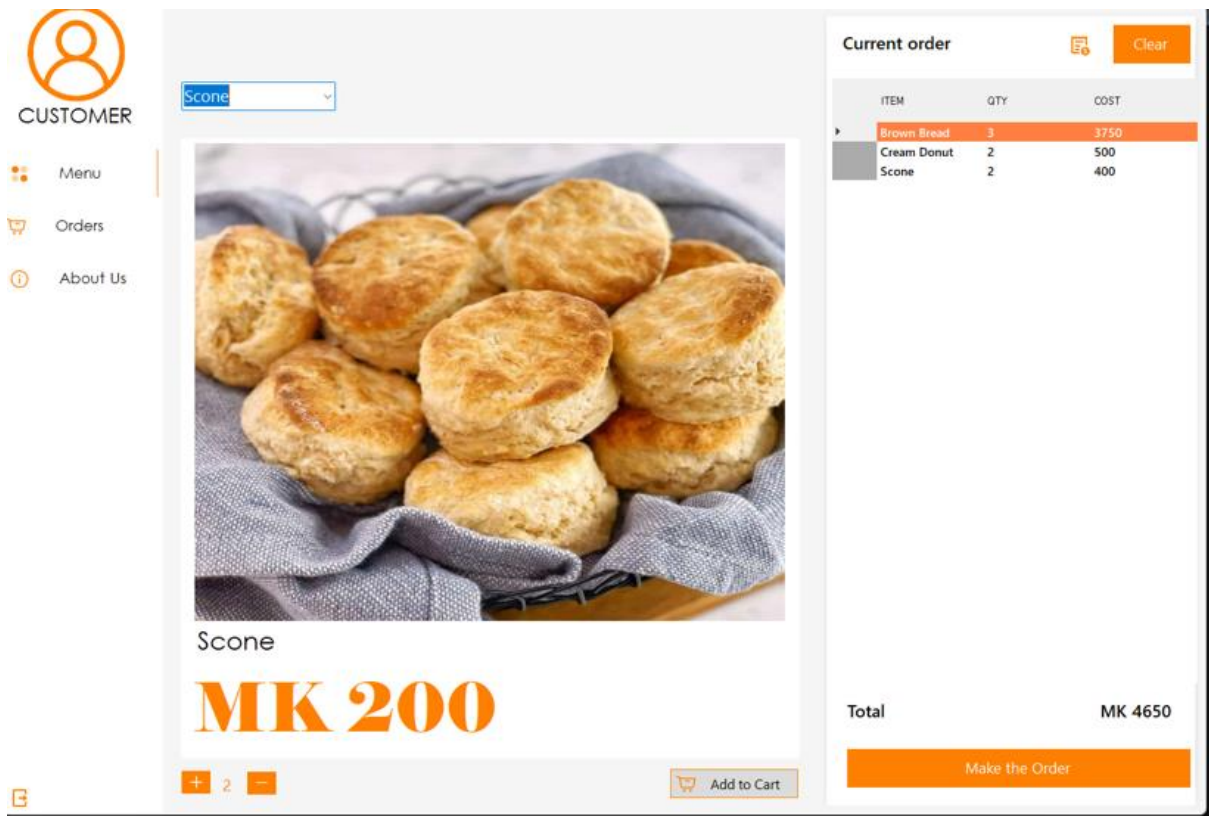
Enter the credentials to log into the system.



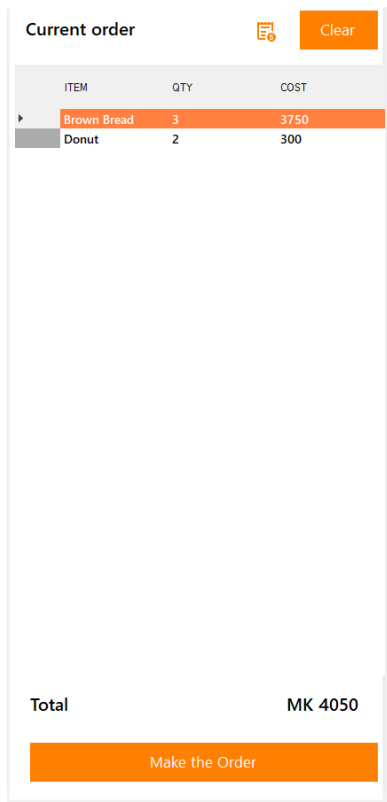
When the customer logs in, they are presented with a menu window.



Choose any of the various products catalogued in a drop down menu



Place orders by selecting the quantity and pressing the add to cart buttons at the bottom of the ordering window





CUSTOMER

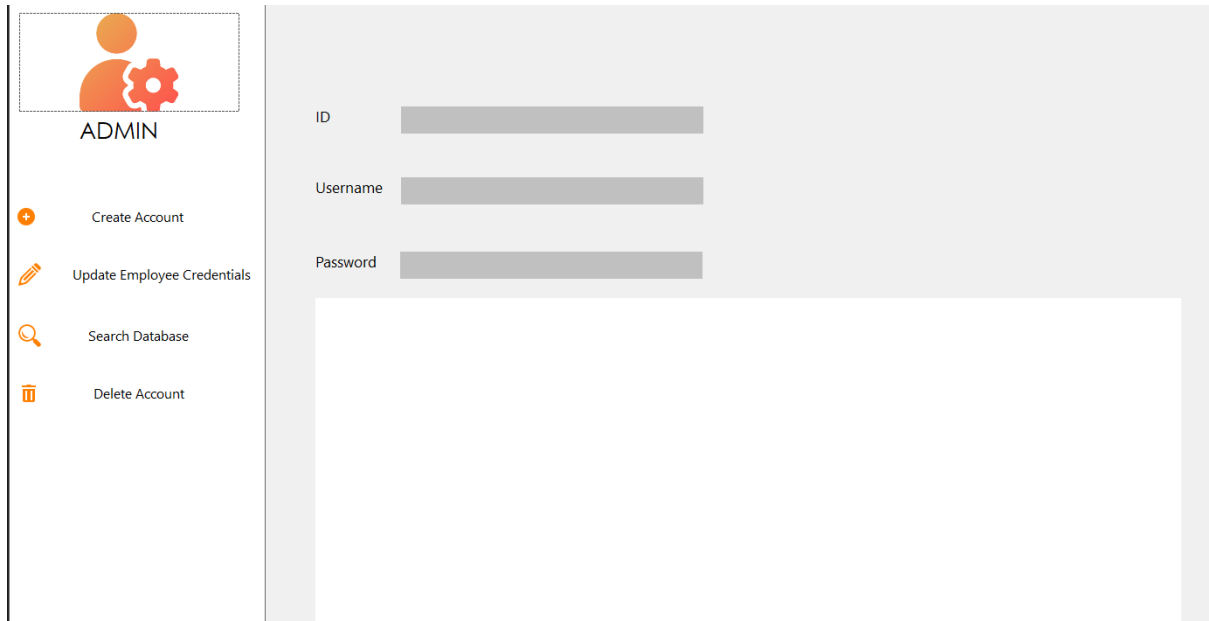
- Menu
- Orders
- About Us

Verify Delivery

ORDER DATE	ITEM	QUANTITY	PRICE	COST	ORDER STATUS
10/9/2023	Scone	2	200	400	pending
10/7/2023	Donut	4	150	600	Received
10/7/2023	Bread Roll	5	350	1750	Received
10/7/2023	Brown Bread	3	1250	3750	pending
10/7/2023	Donut	2	150	300	pending
10/6/2023	Cream Donut	3	250	750	Received
10/5/2023	Scone	3	200	600	Received
10/5/2023	Donut	2	150	300	Received

Should customers wish to view their orders, they can do so by pressed the orders menu bar that is visible at the left hand side of the menu bar

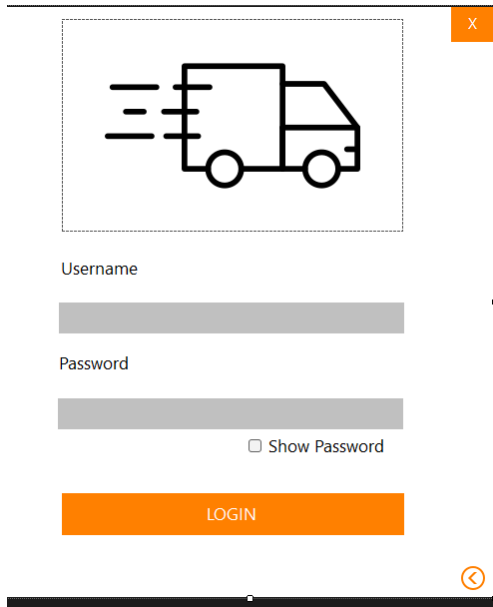
Admin



The screenshot shows an administrative interface. On the left is a sidebar with a header 'ADMIN' and four menu items: 'Create Account' (plus icon), 'Update Employee Credentials' (pencil icon), 'Search Database' (magnifying glass icon), and 'Delete Account' (trash icon). The main area on the right contains three input fields labeled 'ID', 'Username', and 'Password'. Below these fields is a large, empty white rectangular box.

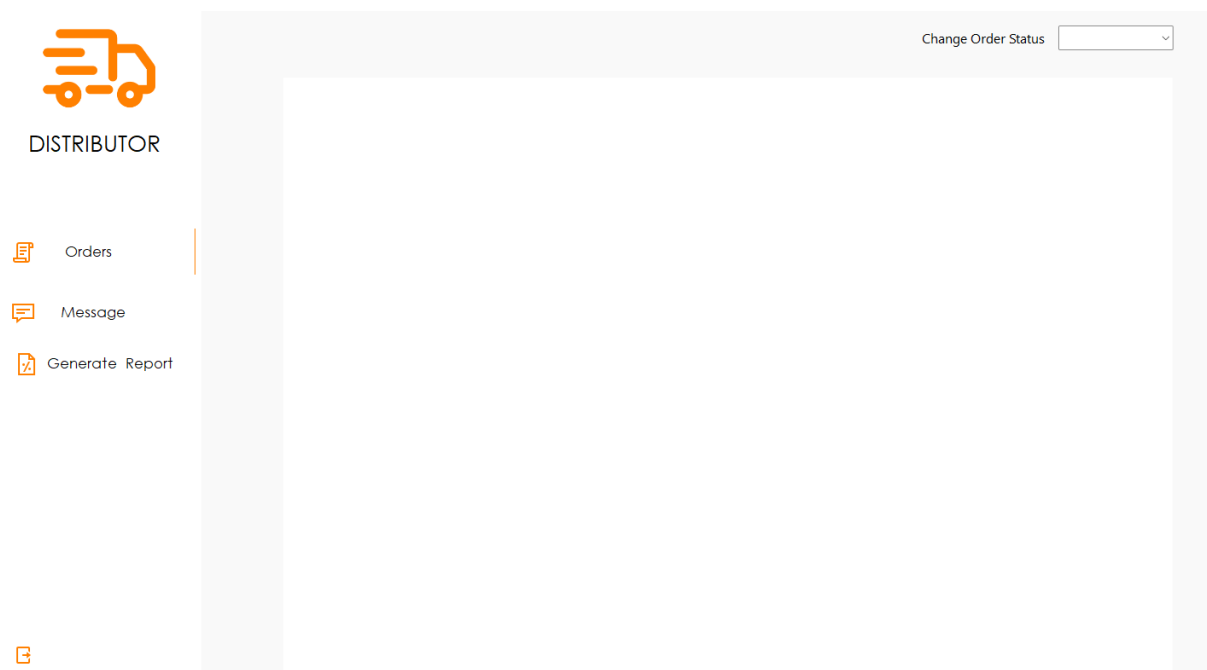
Type in the credentials ADMIN on username and ROOT on password to access the Administrator window. This will allow the management the various accounts of the distributors in the system.

DISTRIBUTORS



A login window for distributors. It features a truck icon in a dashed box at the top. Below the icon are two input fields: 'Username' and 'Password'. The 'Password' field has a 'Show Password' checkbox. At the bottom is an orange 'LOGIN' button. The window has a close button (X) in the top right corner and a back button (left arrow) in the bottom right corner.

Type in DISTRIBUTOR as a username and ROOT as a password to get to the home window of the Distributor section of the system. Here distributors will be presented with a distributor log in window that will ask for them to insert their credentials to log in. Distributors need to have accounts that have been validated by the Administrator to log in.



The distributor home window. It has a header with a truck icon and the word 'DISTRIBUTOR'. On the right side of the header is a 'Change Order Status' dropdown menu. On the left side is a sidebar with three menu items: 'Orders', 'Message', and 'Generate Report', each with a corresponding icon. The main content area is empty.

A distributor window will appear showing all the options available to the distributor



DISTRIBUTOR

Orders

Generate Report

Change Order Status

Order_id	ORDER DATE	ITEM	QUANTITY	PRICE	COST	ORDER STATUS	CUSTOMER ADDRESS
59	10/9/2023	Scone	2	200	400	pending	
60	10/9/2023	Brown Bread	1	1250	1250	pending	
55	10/7/2023	Donut	4	150	600	Received	Chileka Internati...
56	10/7/2023	Bread Roll	5	350	1750	Received	Chitawira, Blanty...
57	10/7/2023	Brown Bread	3	1250	3750	pending	Kameza Lodge
58	10/7/2023	Donut	2	150	300	pending	
54	10/6/2023	Cream Donut	3	250	750	Received	Chitawira, Blanty...
52	10/5/2023	Scone	3	200	600	Received	Shoprite Chichiri
53	10/5/2023	Donut	2	150	300	Received	Ndirande, Blanty...

Press the menu bar to view the orders placed by the customer



DISTRIBUTOR

Orders

Generate Report

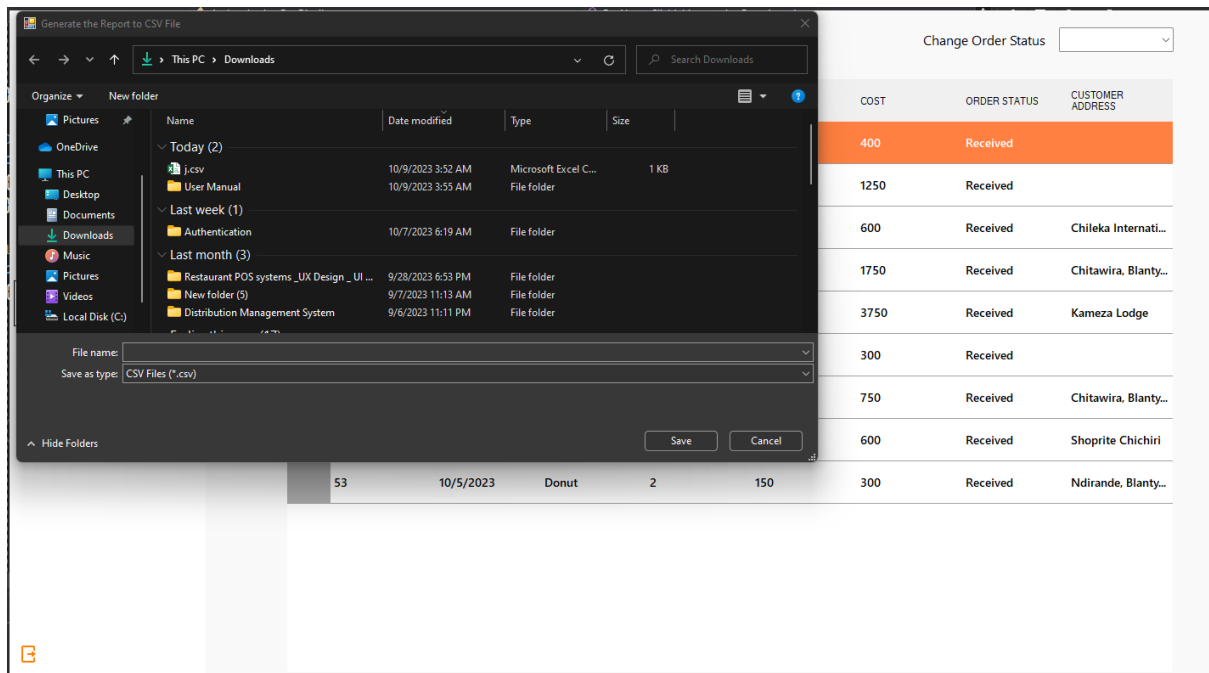
Change Order Status

Order_id	ORDER DATE	ITEM	QUANTITY	PRICE	COST	ORDER STATUS	CUSTOMER ADDRESS
59	10/9/2023	Scone	2	200	400	Received	
60	10/9/2023	Brown Bread	1	1250	1250	Received	
55	10/7/2023	Donut	4	150	600	Received	Chileka Internati...
56	10/7/2023	Bread Roll	5	350	1750	Received	Chitawira, Blanty...
57	10/7/2023	Brown Bread	3	1250	3750	Received	Kameza Lodge
58	10/7/2023	Donut	2	150	300	Received	
54	10/6/2023	Cream Donut	3	250	750	Received	Chitawira, Blanty...
52	10/5/2023	Scone	3	200	600	Received	Shoprite Chichiri
53	10/5/2023	Donut	2	150	300	Received	Ndirande, Blanty...

SMS sent with SID: SM30C13a46da162f6350c630b74971168

OK

When the status of the orders is changed by the distributor, the custom is able to track the status of their orders and they receive a notification message that informs them about the order status. For the customer to receive the SMS, the distributor must have an established network connection.



The distributor needs to press the generate report button which will ask to choose the location the report will be generated

7.6. System Code

```
SaveFileDialog saveFileDialog = new SaveFileDialog();
saveFileDialog.Filter = "CSV Files (*.csv)|*.csv";
saveFileDialog.Title = "Save Quotation to CSV File";

if (saveFileDialog.ShowDialog() == DialogResult.OK)
{
    string csvFilePath = saveFileDialog.FileName;

    string connectionString = @"Data Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=|DataDirectory|\
    string sqlQuery = "SELECT * from orders";

    using (SqlConnection connection = new SqlConnection(connectionString))
    {
        connection.Open();

        using (SqlCommand command = new SqlCommand(sqlQuery, connection))
        {
            using (SqlDataAdapter adapter = new SqlDataAdapter(command))
            {
                DataTable dataTable = new DataTable();
                adapter.Fill(dataTable);
            }
        }
    }
}
```

```

SaveFileDialog saveFileDialog = new SaveFileDialog();
saveFileDialog.Filter = "PDF Files (*.pdf)|*.pdf";
saveFileDialog.Title = "Save Quotation to PDF File";

if (saveFileDialog.ShowDialog() == DialogResult.OK)
{
    string pdfFileName = saveFileDialog.FileName;

    Document doc = new Document();
    try
    {
        PdfWriter writer = PdfWriter.GetInstance(doc, new FileStream(pdfFileName, FileMode.

        doc.Open();
        doc.Add(new Paragraph("Quotation"));

        foreach (DataGridViewRow row in DataGridQuotation.Rows)
        {
            if (!row.IsNewRow)
            {
                string foodItem = row.Cells[0].Value.ToString();
                int quantity = Convert.ToInt32(row.Cells[1].Value);
                float itemTotal = itemcost * quantity;

11 (DataGridQuotation.RowCount == '0')
{
    MessageBox.Show("You do not have any orders", "ORDER PLACEMENT FAILURE", Messag
}
else
{
    SaveDataGridToCSV();
    conn.Open();
    string insert = "INSERT INTO orders (ITEM, QUANTITY, PRICE, [COST], [ORDER STAT
    cmd = new SqlCommand(insert, conn);
    cmd.Parameters.AddWithValue("@Itemname", Itemname.Value);
    cmd.Parameters.AddWithValue("@quantity", quantity.Value);
    cmd.Parameters.AddWithValue("@itemcost", itemcost);
    cmd.Parameters.AddWithValue("@cost", cost.Value);
    cmd.ExecuteNonQuery();
    conn.Close();

    new Geolocation().Show();
    this.Hide();
}

```

```
SelectedValue = ComboBoxFoodItems.SelectedItem.To
```

```
if (SelectedValue == "Brown Bread")
{
    itemcost = 1250;
    brown_Bread1.BringToFront();
}
else if (SelectedValue == "White Bread")
{
    itemcost = 1250;
    white_Bread1.BringToFront();
}
else if (SelectedValue == "Donut")
{
    itemcost = 150;
    donut1.BringToFront();
}
else if (SelectedValue == "Cream Donut")
```

s found

```
DataGridQuotation.Rows.Add(newRow);

decimal sum = 0;

foreach (DataGridViewRow row in DataGridQuotation.Rows)
{
    if (row.Cells[2].Value != null && !string.IsNullOrEmpty(row.Cells[2].Value.ToString()))
    {
        sum += decimal.Parse(row.Cells[2].Value.ToString());
    }
}

LblTotalCost.Text = "MK " + sum;
}
```

```

private void BtnBuy_Click_1(object sender, EventArgs e)
{
    DataGridViewRow newRow = new DataGridViewRow();

    if (ComboBoxFoodItems.Text == "" || LblQuantity.Text == "" + 0)
    {
        MessageBox.Show("Food item or quantity is missing", "CANNOT IM
    }
    else
    {

        Itemname = new DataGridViewTextBoxCell();
        Itemname.Value = ComboBoxFoodItems.Text;
        newRow.Cells.Add(Itemname);

        quantity = new DataGridViewTextBoxCell();
        quantity.Value = LblQuantity.Text;
        newRow.Cells.Add(quantity);

        float itemtotal = itemcost * count;

        cost = new DataGridViewTextBoxCell();
        cost.Value = itemtotal;
        newRow.Cells.Add(cost);
    }
}

```

1 reference

```
private void button3_Click(object sender, EventArgs e)
{
    conn.Open();
    cmd = new SqlCommand("SELECT * FROM Distributors WHERE username = @username ", conn);
    cmd.Parameters.AddWithValue("@Username", txtUsername.Text);
    da = new SqlDataAdapter(cmd);
    DataTable dt = new DataTable();
    da.Fill(dt);
    dataGridViewDistributors.DataSource = dt;
    conn.Close();
}
```

1 reference

```
private void BtnLogOut_Click(object sender, EventArgs e)
{
    new FrmLogIn().Show();
    this.Hide();
}
```

```
private void button2_Click(object sender, EventArgs e)
{
    conn.Open();
    cmd = new SqlCommand("UPDATE Distributors SET username = @Username, password = @password", conn);
    cmd.Parameters.AddWithValue("@ID", TxtID.Text);
    cmd.Parameters.AddWithValue("@Username", txtUsername.Text);
    cmd.Parameters.AddWithValue("@Password", txtPassword.Text);
    cmd.ExecuteNonQuery();
    MessageBox.Show("Account Updated Successfully");
    conn.Close();
}
```

1 reference

```
private void button4_Click(object sender, EventArgs e)
{
    conn.Open();
    cmd = new SqlCommand("DELETE Distributors WHERE Distributor_Id = @ID ", conn);
    cmd.Parameters.AddWithValue("@ID", TxtID.Text);
    cmd.ExecuteNonQuery();
    MessageBox.Show("Account Deleted");
    conn.Close();
}
```

1 reference

```
private void button1_Click(object sender, EventArgs e)
{
    try
    {
        conn.Open();
        cmd = new SqlCommand("INSERT INTO Distributors VALUES (@Username, @Password)", conn);
        cmd.Parameters.AddWithValue("@Username", txtUsername.Text);
        cmd.Parameters.AddWithValue("@Password", txtPassword.Text);
        cmd.ExecuteNonQuery();
        MessageBox.Show("Distributor Account Created Successfully");
        conn.Close();
    }
    catch
    {
        MessageBox.Show("Error, Cannot create account. use another username and password");
    }
}
```

1 reference

0 references

```
class Distributor : User
{
    private int Distributor_id;
    private string username;
    private string password;
}
```

2 references

```
class Location
```

```
{
```

```
    private string customer_address;
```

1 reference

```
    public string Customer_Address
```

```
    {
```

```
        set { customer_address = value; }
```

```
        get { return customer_address; }
```

```
    }
```

```
}
```

```
class Order
```

```
{
```

```
    private int order_id;
```

```
    private string product_id;
```

```
    private DateTime orderddate;
```

0 references

```
    public int Order_ID
```

```
    {
```

```
        set { order_id = value; }
```

```
        get { return order_id; }
```

```
    }
```

0 references

```
    public string Product_ID
```

```
    {
```

```
        set { product_id = value; }
```

```
        get { return product_id; }
```

```
class Product
{
    private int product_id;
    private string product_name;
    private float product_price;
    private int product_quantity;

    0 references
    public int ProductID
    {
        set { product_id = value; }
        get { return product_id; }
    }

    0 references
    public string ProductName
    {
        set { product_name = value; }
        get { return product_name; }
    }

    0 references
    public float ProductPrice
    {
        set { product_price = value; }
        get { return product_price; }
    }
}
```

5 references

class User

{

private int id;

0 references

public int ID

{

set { id = value; }

get { return id; }

}

1 reference

public string Username { set; get; }

1 reference

public string Password { set; get; }

}

References

```
static void GenerateReport(DataTable data, string filePath)
{
    try
    {
        using (StreamWriter writer = new StreamWriter(filePath))
        {
            foreach (DataColumn column in data.Columns)
            {
                writer.Write(column.ColumnName);
                if (column != data.Columns[data.Columns.Count - 1])
                {
                    writer.Write(",");
                }
            }
            writer.WriteLine();

            foreach (DataRow row in data.Rows)
            {
                for (int i = 0; i < data.Columns.Count; i++)
                {
                    writer.Write(row[i]);
                    if (i != data.Columns.Count - 1)
                    {
                        writer.Write(",");
                    }
                }
                writer.WriteLine();
            }
        }
    }
}
```

```

try
{
    if (url.Contains("/place/"))
    {
        int startIndex = url.LastIndexOf("/place/") + 7;
        int endIndex = url.IndexOf("/", startIndex);
        string locationWithPlus = url.Substring(startIndex, endIndex - startIndex);

        string location = locationWithPlus.Replace("+", " ");
        //string shortenedUrl = url.Length > 50 ? url.Substring(0, 50) + "..." : url;
        LblAddress.Text = location;
    }
    else if (url.Contains("@"))
    {
        string[] parts = url.Split('@')[1].Split(',');

        if (parts.Length >= 2)
        {
            LblLatitude.Text = parts[0];
            LblLongitude.Text = parts[1];
        }
    }
    else
    {

```

1 reference

```

private void webView21_SourceChanged(object sender, CoreW
{
    string url = webView21.Source.ToString();
    ExtractAddressFromUrl(url);
}

```

1 reference

```

conn.Open();

string updateSql = $"UPDATE orders SET [ORDER STATUS] = '{selectedStatus}'";
SqlCommand command = new SqlCommand(updateSql, conn);
command.ExecuteNonQuery();
dataGridViewCart.ReadOnly = false;
dataGridViewCart.Refresh();
try
{
    string accountSid = "AC4ec18ae5a2e1f44659161bdb598a1376";
    string authToken = "bd6af5066b52c25dfe6817c2bdebf7d9";
    TwilioClient.Init(accountSid, authToken);
    var toPhoneNumber = new PhoneNumber("+265881836268");
    var fromPhoneNumber = new PhoneNumber("+12565488510");
    string messageText = $"Dear Customer, this is Laher Bakery. We would like to inform you that";
    var message = MessageResource.Create(
        to: toPhoneNumber,
        from: fromPhoneNumber,
        body: messageText
    );
    MessageBox.Show($"SMS sent with SID: {message.Sid}");
}
catch
{
    MessageBox.Show("Internet Connection Error. Please make sure you have an established internet connection.");
}

```